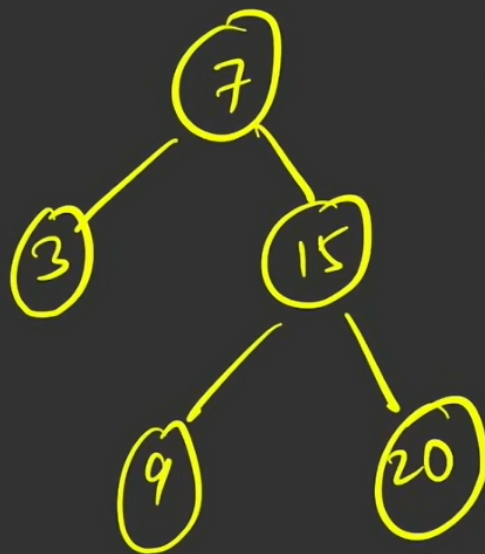


BST iterator



$Sl \rightarrow O(H)$

$TC \approx O(1)$

BSTIterator (7)

next $\rightarrow 3$

next $\rightarrow 7$

hasNext $\rightarrow \text{True}$

next $\rightarrow 9$

hasNext $\rightarrow \text{True}$

next $\rightarrow 15$

hasNext $\rightarrow \text{True}$

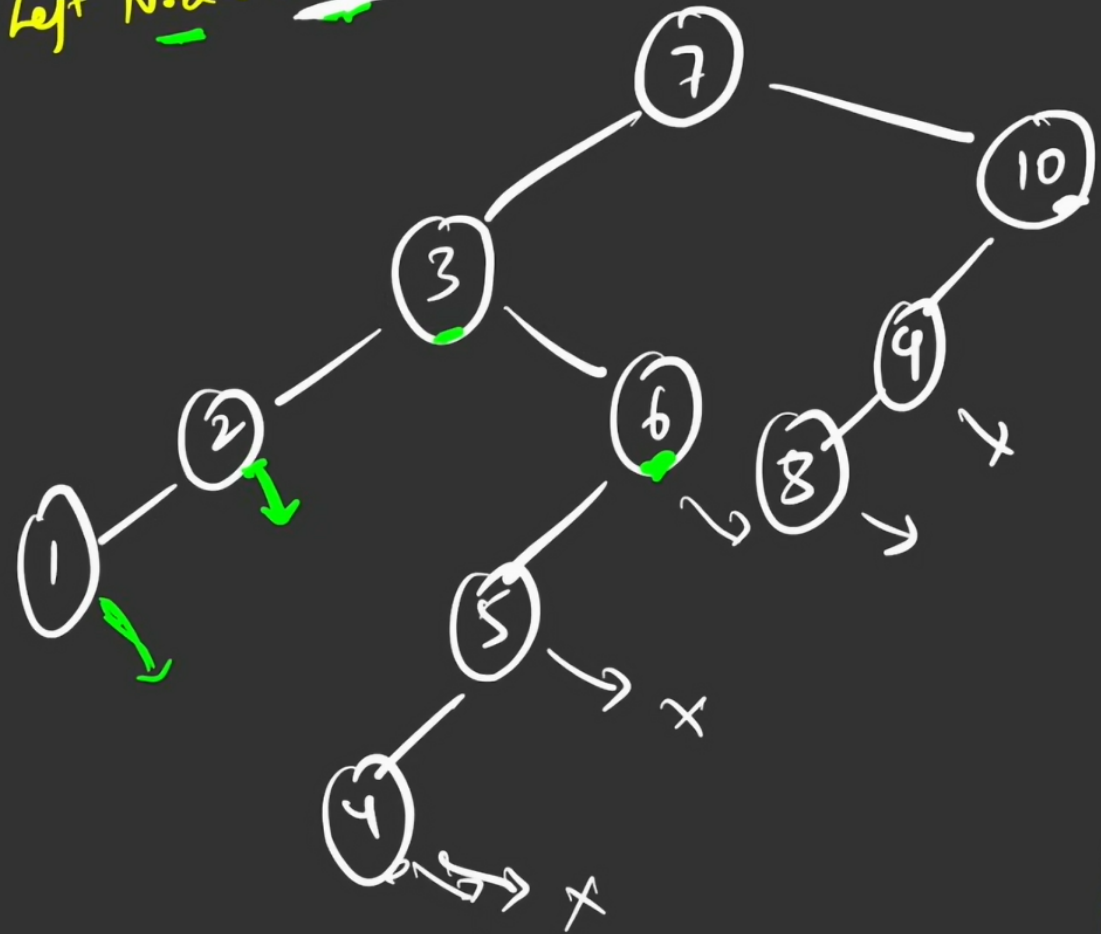
next $\rightarrow 20$

hasNext $\rightarrow \text{False}$



1.50

Left Node Right



BSTIterator(7)
next → 1
next → 2
next → 3
hasNext() → true

4
5
6
+
2
3
7

next → 4
next → 5
next → 6
next → 7
next → 8
next → 9
next → 10

stack → LIFO



```

1  /**
2   * Definition for a binary tree node.
3   * struct TreeNode {
4   *     int val;
5   *     TreeNode *left;
6   *     TreeNode *right;
7   *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
8   *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
9   *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
10  * };
11  */
12  class BSTIterator {
13  private: stack<TreeNode*> myStack;
14  public:
15      BSTIterator(TreeNode* root) {
16          pushAll(root);
17      }
18
19      /** @return whether we have a next smallest number */
20      bool hasNext() {
21          return !myStack.empty();
22      }
23
24      /** @return the next smallest number */
25      int next() {
26          TreeNode* tmpNode = myStack.top();
27          myStack.pop();
28          pushAll(tmpNode->right);
29          return tmpNode->val;
30      }
31
32  private:
33      void pushAll(TreeNode* node) {
34          for (; node != NULL; myStack.push(node), node = node->left);
35      }
36  };
37
38  /**
39   * Your BSTIterator object will be instantiated and called as such:
40   * BSTIterator* obj = new BSTIterator(root);
41   * int param_1 = obj->next();
42   * bool param_2 = obj->hasNext();

```

Your previous code was restored from your local storage. [Reset to default](#)